

Answer any three of the following questions
 Figures in the right hand margin indicate full marks

1.
 - a) Consider the following insertion sort algorithm
Insertion Sort (A) // A is an array of length n
 1. for j = 2 to A.length
 2. key = A[j]
 3. i = j-1
 4. while (i > 0) and (A[i] > key)
 5. A[i+1] = A[i]
 6. i = i-1
 7. A[i+1] = key
 - i) How many comparisons and swaps are needed to sort the next two numbers in [1 3 4 8 9 || 5 2] using insertion sort algorithm. The double bars represent the sort marker. 2
 - ii) What would be the running time of the Insertion sort Algorithm, if all the elements in an input array are equal for example {1, 1, 1, 1, 1, 1}? 1
 - b) Which algorithm will be the best for the following scenario? Justify your answer. 2
You are running a library catalog. You know that the books in your collection are almost in sorted ascending order by title, with the exception of one or two books which are in the wrong place. You want the catalog to be completely sorted in ascending order.
 - c) Consider the following two functions: 3

i) <pre> A() while (n>1) n=n/2 } Assume n ≥ 2 </pre>	ii) <pre> B() { for (i=1; i<=n; i++) for (j=1; j<=n/2; j++) printf("Algorithm"); } </pre>
--	--

Find out the time complexity of both the functions and express those in terms of asymptotic notation. 2
 - d) Use master method to give an asymptotic bound for the following recurrence: 2
 $T(n) = 29T(n/3) + n^3$
2.
 - a) What is the worst-case input for quicksort? Why is it so? Illustrate your answer with a brief example. [Consider an 8-element array of inputs.] 2
 - b) Which one of the following is a heap and which is not? If it is not a heap then (i) state why not, and (ii) make it into a heap by rearranging the values or nodes. 3
 - A = {5, 1, 4, 6, 2}
 - B = {6, 7, 8, 9, 10}
 - C = {22, 13, 46, 12, 11}
 - c) Write down the recurrence equation for the running time of quicksort algorithm in which partition always produces a 9-to-4 proportional split. Solve the recurrence using recursion tree and show that in this case running time is $\Theta(n \lg n)$. 3

- d) Show the steps of inserting the element 27 in the following max-heap 2
 $A = \langle 19, 17, 13, 12, 8, 3, 9, 6, 5 \rangle$

3.

- a) Given a sequence $p = \langle 4, 2, 3, 2, 3 \rangle$ which represents the chain of matrices such that 5
 i -th matrix A_i is of dimension $p_{i-1} \times p_i$. Fill up the m and s tables for computing the cost of matrix chain multiplication using dynamic programming for the matrix chain A .
- b) What is optimal substructure? Give an example of a problem which does not have 2
this property.
- c) What do you mean by longest common subsequence (LCS)? Recursively define the 2
LCS-length of two sequences.
- d) Write down the sequences of four steps to develop a dynamic programming algorithm. 1

4.

- a) Show the steps of the first PARTITION operation of the quick sort algorithm on the 2
array $A = \langle 5, 9, 8, 3, 8, 4, 6, 2, 4 \rangle$. Assume the last element as the pivot.
- b) Consider an array A having same elements like $\langle 7, 7, 7, 7, 7 \rangle$. Can it be considered as 1
a max-heap or a min-heap?
- c) Find a longest common subsequence of the strings "AGGTAB" and "GXTXAYB" by 3
filling up the table for a dynamic programming algorithm.
- d) Do you think that the Longest Common Subsequence problem has overlapping sub 2
problems? Justify your answer using suitable example?
- e) Show that the asymptotic bound of a polynomial of degree n is $O(x^n)$. 2
[Hint: Think about the definition of O -notation and find the values of the constants n_0 and c .]

Ans. to the ques. no 01

[11, 8, 5, 11, 8, 1] (a)

i) The given array,

[1, 3, 4, 8, 9 || 5, 2]

The double bar (||) represents the sort marker means that, the numbers before the marker are already sorted.

The steps to follow to sort the next two numbers of the array are given below.

1) Initialization : Start with the number 5 (position after the sort marker)

key = 5, j = 6

2) Comparison and swap :

Compare the numbers before 5 and move larger numbers to the right. Insert 5 in the correct position.

3) Insert Key : After inserting key = 5, the array becomes [1, 3, 4, 5, 8, 9 || 2]

4) Repeat the same process with the number 2.

Result : The array becomes $[1, 2, 3, 4, 5, 8, 9, 11]$

The numbers 5 and 2 required a total of 4 comparisons and 3 swaps to be sorted using the insertion sort algorithm.

ii)

If all the elements in an input array are equal like $\{1, 1, 1, 1, 1, 1\}$ the running time of the insertion sort algorithm will still be $O(n^2)$.

In the worst case scenario, in insertion sort has to compare and shift each element to its correct position in the sorted portion of the array.

Even though the elements are equal, the algorithm would still need to perform the comparisons and shifts for each element and the running time will be the same.

(b)

In scenarios, where the data is almost sorted or partially sorted, the insertion sort algorithm tends to perform very well.

Given that the books in the library catalog are almost sorted in ascending order by title, Insertion sort can efficiently handle this situation.

Because it has an average and best time complexity of $O(n)$ for already sorted or nearly sorted data.

Insertion sort works by gradually building the sorted portion of the array (catalog) by repeatedly selecting elements from the unsorted portion and inserting them into their current position within the sorted portion.

Since most of the books are already in correct order, the number of comparisons and shifts required will be relatively small.

1(c)
(i)

Given that,

A() {
 while (n > 1)
 n = n/2
}

Assume $n \geq 2$

Time complexity = $O(\log_2 n)$.

(ii)

Given that,

B() {
 for (i = 1; i <= n; i++)
 for (j = 1; j <= n/2; j++)
 print ("Algorithm");
}

Time complexity = $O(n * \frac{n}{2})$
= $O(\frac{n^2}{2})$
= $O(n^2)$

(Ans).

$$\underline{\underline{1(d)}}$$

Given recurrence —

$$T(n) = 29T(n/3) + n^3 \quad \text{--- (i)}$$

We know that,

$$T(n) = aT(n/b) + O(n^k \log^p n) \quad \text{--- (ii)}$$

Comparing (i) and (ii) we get $\Rightarrow$

$$a = 29, \quad b = 3, \quad k = 3, \quad p = 0$$

Here, $\frac{\log a}{\log b} = \frac{\log 29}{\log 3}$

$$b^k = 3^3 = 27 < a$$

$$\begin{aligned} T(n) &= O(n^{\log_b a}) \\ &= O(n^{\log_3 29}) \end{aligned}$$

(Ans)

Answer No- 2

a) What is the worst-case input for quick-sort? why is it so? Illustrate your answer with a brief example -

Answer: In quick Sort, worst Case occurs when the pivot element is either greatest or smallest element.

Suppose, if the pivot element is always the last element of the array, the worst-case would occur when the given array is sorted already in ascending or descending order.

The worst-case time complexity of quicksort is $O(n^2)$.

Worst Case will happen when array is sorted.

The worst occurs in the following cases -

1) Array is already sorted in the same order.

$$A = (33, 45, 56, 87, 111, 221, 555)$$

2) Array is already sorted in reverse order.

$$A = (555, 221, 111, 87, 56, 45, 33)$$

Quick Sort is

2(a) ~~A = heap~~ $\{5, 11, 4, 6, 2\}$

$A = \{5, 1, 4, 6, 2\}$

~~B =~~ $B = \{6, 7, 8, 9, 10\}$

$C = \{22, 13, 46, 12, 11\}$

2(b) Which one of the following is a heap and which is not? if not a heap then

i) state why not? ii) Make it into a heap by rearranging the values or nodes.

Solution: i) A heap is a specialized tree-based data structure that satisfies the heap property. There are two common types of heaps: max heaps and min heaps. In a max heap, for any given node, the value of the node is greater than or equal to the values of its children. In a min heap, the value of the node is less than or equal to the values of its children.

Let's analyze each of the sets (A, B and C) to

determine whether they are heaps or not based on the max heap property.

Set A: $\{5, 1, 4, 6, 2\}$

This set does not satisfy the max heap property because the root node (5) is less than one of its children (6). Therefore, set A is not a max-heap.

Set B: $\{6, 7, 8, 9, 10\}$

This set satisfies the max heap property. In every subtree, the parent node's value is greater than or equal to the values of its children. Therefore, set B is a max heap.

Set C: $\{22, 13, 46, 12, 11\}$

This set does not satisfy the max heap property. For example, the parent node with value 22 has a child with value 46, violating

the max heap property. Therefore, set C is not a max heap.

If you need to transform sets A and C into max heaps, you would need to perform heapify operations to rearrange the elements and ensure that the max heap property is satisfied.

ii) A heap is a specific type of binary tree that satisfies the heap property, which depends on whether it's a min-heap or a max-heap. In a min-heap, for any given node, its parent node has a value smaller than or equal to the node's value. In a max-heap, the parent node's value is greater than or equal to the node's value. Let's analyze each of the given sets to determine

Whether they are heaps or not:

1. Set $A = \{5, 1, 4, 6, 2\}$

This set is not a heap since the parent-child relationships and values do not satisfy the heap property. To transform it into a

min-heap, you need to rearrange the value.

Rearrange: $A = \{1, 2, 4, 6, 5\}$

2. Set $B = \{6, 7, 8, 9, 10\}$

Similar to the previous set, this is not a heap. To transform it into a max-heap, you need to rearrange the values.

Rearrange: $B = \{10, 9, 8, 7, 6\}$

3. Set $C = \{22, 13, 46, 12, 11\}$

Just like the previous sets, this set is not

a heap. To transform it into a max-heap, you need to rearrange the value.

Rearrange: $C = \{46, 22, 13, 12, 11\}$

After this rearrange set is a max-heap because the heap property is satisfied.



(c)

Quick Sort is a popular sorting algorithm that has an average case time Complexity of $(n \log n)$. However, in the worst-case, the time Complexity can be as high as $O(n^2)$. One way to improve the worst case behaviour of quick sort is to use a proportional split when partitioning the array. This means that the partition divides the array into two sub arrays whose sizes are in a fixed proportion to each other.

In this case, we consider a 9 to 4 proportional split, which means that one partition is $9/13$ of the original size and the other is $4/13$. Using this split we can write the recurrence

equation for the running time of

quick sort —

$$T(n) = T(9n/13) + T(4n/13) + O(n)$$

Here,

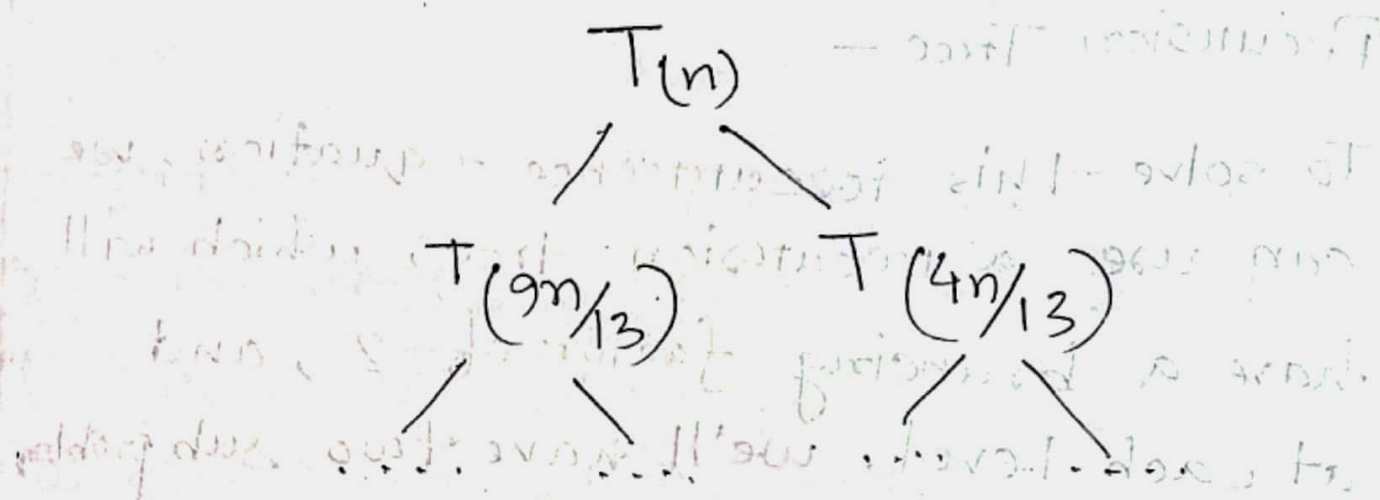
$T(n)$ represents the running time of quick sort on an input of size n . The first term, $T(9n/13)$, represents the time taken to sort the larger partition ($9n/13$ elements), and the second term, $T(4n/13)$, represents the time taken to sort the smaller partition ($4n/13$ elements).

The last term, $O(n)$, corresponds to the time taken for the partitioning step.

Recursion Tree -

To solve this recurrence equation, we can use a recursion tree, which will have a branching factor of 2, and at each level, we'll have two sub problems of size $9n/13$ and $4n/13$. The tree will have a depth of $\log_{13/9}(n)$ since the smaller of subproblem size ~~de~~ decreases by a factor of $13/9$ in each level.

the total work done at each level is $O(n)$, as each level involves partitioning the array and combining the results from the subproblems. Therefore, the total work done at each level of the recursion tree is proportional to n .



Since the work at each level is $O(n)$, and the tree has $\log_{13/9} n$ levels in the tree, the total work done is $O(n * \log_{13/9} n)$.

Hence, the running time of the Quicksort algorithm with a partition that always produces a 9-to-4 proportional split is indeed $\Theta(n \log n)$, meaning the algorithm's average and worst-case time complexity is proportional to $n \log n$.

2 (d) Show the steps of inserting the element 27 in the following max-heap $A = \{19, 17, 13, 12, 8, 3, 9, 6, 5\}$

Solution:

Certainly! To insert the element 27 into the given max-heap A , you would follow these steps:

1. Start by adding the new element 27 at the end of the heap (as the last leaf node).

Max-Heap $A: \{19, 17, 13, 12, 8, 3, 9, 6, 5, 27\}$

2. Now, we need to ~~store~~ restore the max-heap property by "bubbling up" the newly inserted element if it violates the heap property.

Compare the newly inserted element (27) with its parent node (which is 5):

Since $27 > 5$, we need to swap them.

Max-Heap A after the first swap: $\{19, 17, 13, 12, 8, 3, 9, 6, 27, 5\}$

3. Continue comparing and swapping the element with its parent until the max-heap property is ~~not~~ restored.

Now compare 27 with its new parent (19):
Since $27 > 19$, no further swaps are needed.

The max-heap property is restored.

Max-Heap A after the insertion of 27: $\{27, 17, 19, 12, 8, 3, 9, 6, 13, 5\}$.

The final heap is a valid max-heap with the element 27 inserted while maintaining the max-heap property.

3(a) Given a sequence $p = (4, 2, 3, 2, 3)$ which represents the chain of matrices such that i -th matrix A_i is of dimension $p_{i-1} \times p_i$, fill up the m and s tables for computing the cost of matrix chain multiplication using dynamic programming for the matrix chain A .

Ans: Here $m[i][j]$ will represent the minimum cost of multiplying matrices from $A[i]$ to $A[j]$ and $s[i][j]$ will represent the index at which the split between matrices should be made to achieve that minimum cost.

Given the sequence $p = (4, 2, 3, 2, 3)$; we have 5 matrices: A_1 with dimensions 4×2 , A_2 with dimensions 2×3 , A_3 with dimensions 3×2 , A_4 with dimensions 2×3 and A_5 with dimensions 3×3 .

(2)
~~2~~
Let's initialize the tables and then fill them using dynamic programming.

1. Initialize m and s tables with zeros.

2. For chain of length 1 from 2 to n (total number of matrices):

- For each i from 1 to $n - 1 + 1$.
- Calculate $j = i + 1 - 1$.
- Initialize $m[i][j]$ with a large value.
- For k from i to $j - 1$.
- Calculate $q = m[i][k] + m[k+1][j] + p[i-1] * p[k] * p[j]$.
- If q is smaller than $m[i][j]$, update $m[i][j]$ with q and $s[i][j]$ with k .

After filling the tables, the m table will store the minimum cost of matrix chain multiplication and the s table will store the optimal split points.

Hence the filled m and s tables for the given sequence:

m Table:

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 24 & 30 & 60 \\ 0 & 0 & 0 & 12 & 30 \\ 0 & 0 & 0 & 0 & 18 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$
s Table:

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 2 & 2 \\ 0 & 0 & 0 & 0 & 3 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

In the m table the value $m[1][5] = 60$ represents the minimum cost of multiplying matrices from A_1 to A_5 . The s table indicates that the optimal split point for multiplying matrices A_1 to A_5 is between A_1 and A_2 (at index 1).

This dynamic programming approach helps us efficiently find the optimal way to multiply

matrices in the chain while minimizing the total number of scalar multiplications.

3(b) What is optimal substructure? Give an example of a problem which does not have this property.

Ans: Optimal substructure is a concept in dynamic programming and algorithm design that describes a property of problems where the optimal solution to the overall problem can be constructed by combining the optimal solutions of its smaller subproblems. In a problem with optimal substructure, we can often break down the larger problem into smaller, overlapping subproblems that can be solved independently, and then combine their solutions to obtain the optimal solution for the entire problem. This property is crucial for the effectiveness of many dynamic programming algorithms, as

it allows for efficient recursive solution methods.

Let's consider the Longest Increasing Subsequence (LIS) problem as an example of a problem that lacks the optimal substructure property.

In the LIS problem, we are given a sequence of numbers, and the goal is to find the length of the longest subsequence in which the elements are in increasing order.

Here's an example sequence of numbers:

Sequence: 10, 22, 9, 33, 21, 50, 41, 60, 80.

A valid increasing subsequence in the sequence would be: 10, 22, 33, 50, 60, 80 with a length of 6.

The reason the LIS problem lacks optimal substructure is that the optimal solution for the entire problem cannot be directly constructed from the optimal solutions

of its subproblems.

Specifically, the longest increasing subsequence of a subsequence might not involve the longest increasing subsequence of its smaller subsequences.

For example, consider the subsequence 10, 9, 21, 41. The longest increasing subsequence within this subsequence is 10, 21, 41, with a length of 3. However, if we take the entire sequence into account, we see that the longest increasing subsequence is longer (length 6).

The lack of optimal substructure makes solving the LIS problem a bit more complex and it often requires specialized ~~tech~~ techniques like dynamic programming or binary search-based algorithms to find the longest increasing subsequence efficiently.

Question (C) $\Rightarrow$ What do you mean by longest common subsequence (LCS)? Recursively define the LCS-length of two sequences.

$\Rightarrow$ The Longest Common Subsequence (LCS) is a fundamental concept in computer science and is used to find the longest subsequence that is common to two sequences. A ^{sub}sequence is a sequence that can be derived from another sequence by deleting some or no elements without changing the order of the remaining elements.

Let's define the LCS-length of two sequences

A & B , recursively:

i) Base Case: If either of the sequences A or B is empty, then the LCS-length is 0, i.e. $LCS(A, \emptyset) = LCS(\emptyset, B) = 0$

ii) Recursive Case: If the last elements of both sequences A and B are the same, then the LCS-length of A and B is one plus the LCS-length of their prefixes with the last elements removed,

Mathematically,

• If $A[m] = B[n]$, then $LCS(A, B) = 1 + LCS$

$(A[1:m-1], B[1:n-1])$, where $A[1:m-1]$ represents the sequence A without its last element, and $B[1:n-1]$ represents the sequence B without its last element.

iii) If the last elements of sequences A & B are different, then LCS -length is the maximum of the LCS -length of the two possible cases where we remove the last element from either sequence A or sequence B . Mathematically:

• If $A[m] \neq B[n]$, then $LCS(A, B) = \max(LCS(A[1:m-1], B), LCS(A, B[1:n-1]))$.

By recursively applying these rules, you can compute the LCS -length of two sequences A and B . The actual LCS can also be constructed.

using dynamic programming techniques by storing the intermediate results in a table.

For example, consider the sequence $A = "AGGTAB"$ and $B = "GXTXAYB"$. The LCS-length of A and B can be calculated as follows:

$$\begin{aligned} \text{LCS}(A, B) &= \text{LCS}("AGGTA", "GXTXAY") + 1 \quad (\text{since the last elements are the same}) \\ &= \max(\text{LCS}("AGGTA", "GXTXAY"), \text{LCS}("AGGTAB", "GXTXA")) + 1 \\ &= \max(\text{LCS}("AGGT", "GXTXAY"), \text{LCS}("AGGTA", "GXTX")) + 1 \end{aligned}$$

$$\begin{aligned} &= \max(\max(\text{LCS}("AGGT", "GXTXAY"), \text{LCS}("AGGT", "GXTX")), \max(\text{LCS}("AGGTA", "GXTX"), \text{LCS}("AGGT", "GXTX")) + 1 \end{aligned}$$

$$= \max(2, 1) + 1$$

$$= 3$$

Therefore, the LCS-length of "AGGTAB" and "GXTXAYB" is 3, and the LCS itself could be "GTAB".

d) Write down the sequences of four steps to develop a dynamic programming algorithm.

⇒ Developing a dynamic programming algorithm involves breaking down a complex problem into smaller subproblems and using memorization or tabulation to avoid redundant computations.

Here are the four key steps to develop a dynamic programming algorithm:

1) Characterize the structure of an optimal solution:

- Understand the problem and the nature of the optimal solution.
- Determine the decision points or choices that lead to the optimal solution.
- Identify the subproblems that need to be solved to reach the final solution.

2) Define the Recurrence Relation:

- Express the relationship between original problem and its subproblems.

- Formulate a recurrence relation that relates the optimal solutions of its subproblems.
- The recurrence relation should be based on the optimal decisions made at each step and should define how to combine the solutions of smaller subproblems to solve the larger problem.

3) Design the Dynamic Programming Table or Memoization Structure:

- Determine the dimensions of the dynamic programming table or memoization structure.
- Decide how to index the table based on the variables and parameters involved in the problem.
- Initialize the table with appropriate values, usually starting with the base cases derived from the smallest subproblems.

4) Fill in the Table or Implementation Memoization:

- Iterate over the table in a systematic manner, solving subproblems in a bottom-up fashion (tabulation) or using memoization (top-down approach with recursion and caching).

• Apply the recurrence relation to compute the values of larger subproblems based on the solutions of smaller subproblems.

• Make use of the stored values in the table or memoization structure to avoid redundant computation.

• Depending on the problem, you might need to return the final solution by combining the values stored in the table in a specific way.

3) Design the Dynamic Programming Table or Memoization Structure:

- Determine the dimensions of the dynamic programming table or memoization structure.
- Decide how to index the table based on the variables and parameters involved in the problem.
- Initialize the table with appropriate values, usually starting with the base case defined from the smallest subproblems.
- Fill in the table or memoization structure.
- Iterate over the table in a systematic manner, solving subproblems in a bottom-up fashion (tabulation) or using recursion and caching (memoization).

4

a) Ques: Show the steps of the first PARTITION operation of the quick sort algorithm on the array $A = \{5, 9, 8, 3, 8, 4, 6, 2, 4\}$

Ans: $A = \{5, 9, 8, 3, 8, 4, 6, 2, 4\}$

a)

1	2	3	4	5	6	7	8	9
5	9	8	3	8	4	6	2	4

pivot
→ 4
 $j=1, i=0$
 $(5 > 4)$

b)

1	2	3	4	5	6	7	8	9
5	9	8	3	8	4	6	2	4

 $j=2, i=0$
 $(9 > 4)$

c)

1	2	3	4	5	6	7	8	9
5	9	8	3	8	4	6	2	4

 $j=3, i=0$
 $(8 > 4)$

d)

1	2	3	4	5	6	7	8	9
5	9	8	3	8	4	6	2	4

 $j=4, i=1$
 $(3 < 4)$
Exchange $A[1]$ and $A[4]$

e)

1	2	3	4	5	6	7	8	9
3	9	8	5	8	4	6	2	4

 $j=5, i=1$
 $(8 > 4)$

f)

1	2	3	4	5	6	7	8	9
3	9	8	5	8	4	6	2	4

 $j=6, i=2$
 $(4 = 4)$
Exchange $A[2]$ & $A[6]$

g)

1	2	3	4	5	6	7	8	9
3	4	8	5	8	9	6	2	4
p	i					j		r

$j=7, i=2$
(6 > 4)

h)

1	2	3	4	5	6	7	8	9
3	4	8	5	8	9	6	2	4
p		i				j		r

$j=8, i=3$
(2 < 4)

Exchange $A[3]$ &
 $A[8]$

i)

1	2	3	4	5	6	7	8	9
3	4	2	5	8	9	6	8	4
p			i					j

$j=9, i=4$

exchange

$A[4]$ & $A[9]$

j)

1	2	3	4	5	6	7	8	9
3	4	2	4	8	9	6	8	5

4 b) Ques:- Consider an array A having same element like $(7, 7, 7, 7, 7)$. Can it be considered as a max-heap or a min-heap?

Ans:- In a max-heap, every parent node has a value greater than or equal to its children nodes. In a min heap, every parent node has a value less than or equal to its children nodes.

Array A, which contains the elements $(7, 7, 7, 7, 7)$, we can see that all the elements are the same.

However, this array cannot be considered a max-heap or a min-heap because it doesn't satisfy the properties of either type of heap.

For a max-heap:-

- * The root element should be maximum element in the heap.

For a min heap:-

- * The root element should be the minimum element in the heap.

Since all the elements in the array are the same (7), neither of these heap properties are satisfied.

Therefore, the array cannot be considered as a max-heap or a min-heap.

4) Ques:- Find a longest common subsequence of the string "AGGITAB" and "GIXTXAYB" by filling up the table for a dynamic programming algorithm.

Ans:-
 $X = \{A, G, G, T, A, B\}$
 $Y = \{G, X, T, X, A, Y, B\}$

	Y_i	A	G	G	T	A	B
X_i	0	0	0	0	0	0	0
G	0 [↑]	0 [←]	1 [↑]	1 [↑]	1 [←]	1 [←]	1 [←]
X	0 [↑]	0 [↑]	1 [↑]	1 [↑]	1 [←]	1 [←]	1 [←]
T	0	0 [↑]	1 [↑]	1 [↑]	2 [↑]	2 [←]	2 [←]
X	0	0 [↑]	1 [↑]	1 [↑]	2 [↑]	2 [←]	2 [←]
A	0	1 [↑]	1 [←]	1 [←]	2 [↑]	3 [↑]	3 [←]
Y	0	1 [↑]	1 [←]	1 [←]	2 [↑]	3 [↑]	3 [↑]
B	0	1 [↑]	1 [←]	1 [←]	2 [↑]	3 [↑]	4 [↑]

LCS = BATG

= ABGT

4
d) Ques:- Do you think that the longest common subsequence problem has overlapping sub problems? Justify your answer using suitable example?

Ans:- Let us consider two strings be $x = \{x_1, x_2, \dots, x_m\}$ and $y = \{y_1, y_2, \dots, y_n\}$. To find LCS of x and y , we should examine either one or two sub-problems.

If $x_m = y_n$, we must find an LCS of x_{m-1} and y_{n-1} . Appending $x_m = y_n$ to this LCS yields LCS of x and y .

If $x_m \neq y_n$, then we must solve two subproblems:

1. finding an LCS of x_{m-1} and y
 2. finding an LCS of x and y_{n-1}
- whichever of those two LCS is longer is an LCS of x and y . Because these cases exhaust all possibilities.

we can readily see the overlapping sub-problem properly in the LCS problem. To find an LCS of X and Y , we may need to find the LCS of X and Y_{n-1} and of X_{m-1} and Y . But each of these sub-problems has the subproblems of finding an LCS of X_{m-1} and Y_{n-1} . Many other subproblems share subproblems.

e) Ques:- Show that the asymptotic bound of a polynomial of degree n is $O(n^n)$

Ans:- Given a nonnegative integer d , a polynomial in n of degree d is a function $p(n)$ of the form

$$p(n) = \sum_{i=0}^d a_i n^i$$

Where the constants $a_0, a_1, \dots, a_d$ are the coefficients of the polynomial and $a_d \neq 0$

A Polynomial is asymptotically $p(n)$ of degree d , we have $p(n) = O(n^d)$. For any real constant $a \geq 0$, the function n^a is monotonically increasing, and for any real constant $a \leq 0$, the function n^a is monotonically decreasing. We say that a function $f(n)$ is polynomially bounded if $f(n) = O(n^k)$ for some constant k .